

Multi-Agent & Kết Nối Hệ Thống

AICB-P1 · Ngày 9 · MCP, A2A & LangGraph

Giảng Viên: Ngô Anh Tuấn

- Sr. MLOps Engineering - Techcombank
- Lead MLOps Team - Obello

VinUniversity · Phase 1 · Tuần 2 · 2026

HÃY SUY NGHĨ...



*"Bạn có 1 agent rất giỏi. Nhưng bài toán đã quá lớn cho 1 agent.
Làm thế nào để hệ thống vẫn rõ vai trò, dễ kiểm soát, và dễ mở rộng?"*

Giữ câu hỏi này trong đầu khi học bài hôm nay

Lộ Trình 9 Ngày Đã Đi Đến Đây



Day 08 đã dạy cách lấy đúng thông tin. Day 09 hỏi câu tiếp theo: khi bài toán lớn hơn một agent, ta tổ chức hệ thống như thế nào?

Nội Dung Bài Học

1. Giới hạn của single-agent
2. Mental model: tư duy hệ thống
3. Multi-agent patterns
4. Supervisor-worker deep dive
5. Shared state & message contract

6. MCP — chuẩn kết nối tool
7. A2A — giao tiếp giữa agents
8. Orchestration với LangGraph
9. Observability & debugging
10. Lab 9 + deliverable

Mục Tiêu Ngày 9

- Giải thích được vì sao single-agent bắt đầu quá tải khi bài toán cần nhiều vai trò và nhiều nguồn lực
- Phân biệt được các pattern supervisor-worker, pipeline, debate, và hierarchical — và biết chọn đúng pattern
- Hiểu MCP là chuẩn nối agent với tool / service bên ngoài, và A2A là cách agents giao việc cho nhau với message contract rõ
- Dùng LangGraph để hình dung graph, state, và conditional routing trong hệ multi-agent
- Thiết kế được trace & observability để debug và cải thiện hệ thống
- Nâng cấp artifact Day 08 thành hệ thống Supervisor + Workers có trace rõ ràng

Deliverable Cuối Ngày

Bản nâng cấp từ Day 08 gồm supervisor, 2–3 workers, 1 kết nối tool qua MCP, và trace log cho toàn bộ luồng phối hợp

- 1 supervisor nhận task, route đúng worker, và tổng hợp kết quả
- 2–3 worker chuyên vai trò như retrieval, tool use, synthesis
- 1 kết nối external capability qua MCP
- 1 trace dễ đọc để giải thích agent nào đã làm gì và khi nào

01

Từ Single-Agent Sang Multi-Agent

Day 08 giúp agent biết retrieve và trả lời grounded; Day 09 trả lời câu hỏi khi nào một agent không còn đủ để gánh toàn bộ bài toán

Từ Artifact Day 08 Sang Bài Toán Lớn Hơn

Day 08 đã làm được

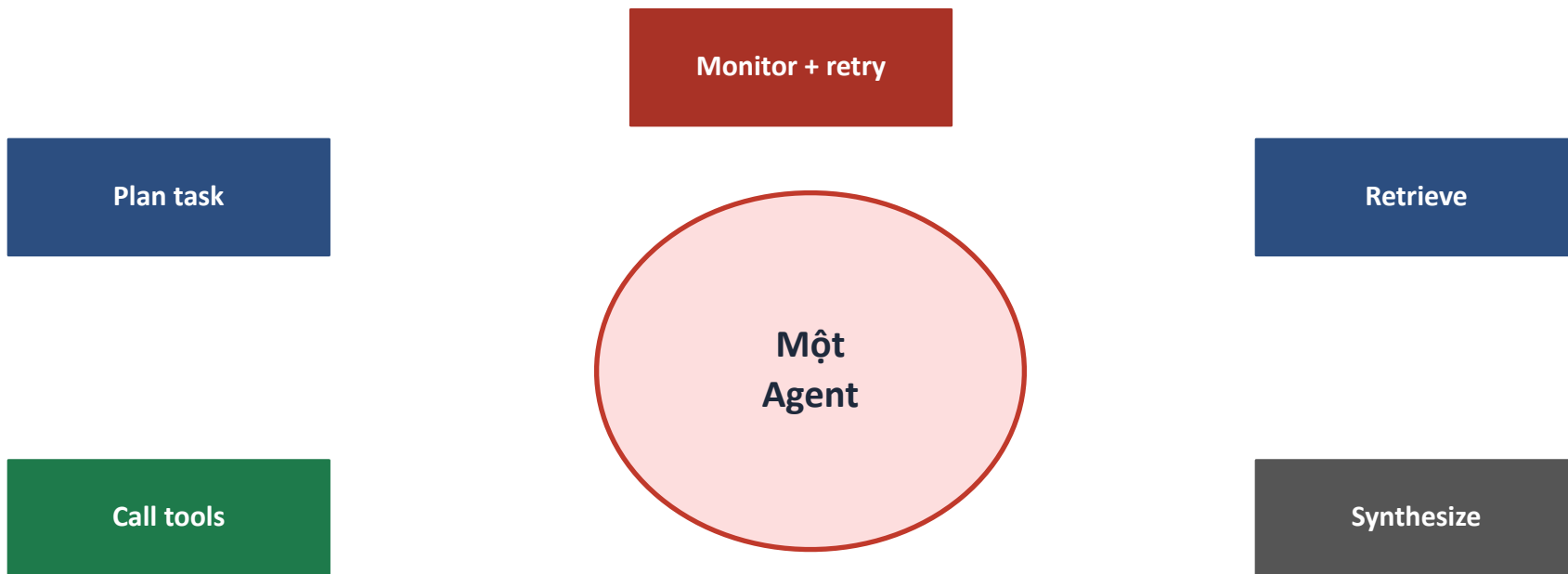
- retrieve đúng tài liệu hơn
- rerank hoặc lọc context tốt hơn
- generate câu trả lời grounded hơn

Nhưng khi hệ thống lớn lên

- phải phân tích task trước khi retrieve
- phải gọi thêm tool ngoài
- phải chia việc và tổng hợp nhiều kết quả
- phải theo dõi trace để debug

Day 09 không phủ định Day 08. Nó là bước tiếp theo: biến một agent có RAG thành một hệ thống có vai trò, có phối hợp, và có điểm mở rộng rõ ràng.

Khi Một Agent Bắt Đầu Quá Tải



Một nơi phải làm quá nhiều việc sẽ khó tối ưu, khó debug, và khó scale.

4 Giới Hạn Cốt Lõi Của Single-Agent

1. Context bottleneck

Một agent phải giữ quá nhiều mục tiêu, tool outputs, evidence, và state. Context window có giới hạn cứng.

2. Specialization trade-off

Agent càng ôm nhiều vai, prompt càng dài và khó ổn định. Giải đều mọi thứ thường đồng nghĩa với không thật sự giỏi vai nào.

3. Parallelism hạn chế

Một agent thường chạy tuần tự. Khi có nhiều việc độc lập, hệ thống vẫn phải chờ từng bước — tăng latency không cần thiết.

4. Reliability yếu

Nếu agent chọn sai tool hoặc hiểu sai task ở đầu luồng, toàn bộ hệ thống dễ đi lệch theo. Không có isolation để khoanh vùng lỗi.

Thực Tế: Context Window Bottleneck Trông Như Thế Nào?

Kịch bản thực tế

- Agent nhận task: phân tích hợp đồng 80 trang + tra cứu luật + tóm tắt rủi ro
- Tool call trả về 12,000 tokens
- Chat history đã chiếm 6,000 tokens
- Prompt gốc: 3,000 tokens
- Còn lại cho reasoning: $\approx 3,000$ tokens

Lưu ý: Agent bắt đầu "quên" phần đầu tài liệu khi xử lý phần cuối
— lost-in-the-middle problem.

Dấu hiệu nhận biết

- câu trả lời thiếu thông tin ở giữa document
- agent lặp lại bước đã làm
- reasoning ngắn bất thường ở bước cuối
- tool call với empty context

Dấu Hiệu Nên Nghĩ Tới Multi-Agent

- ✓ Task có nhiều bước vai trò khác nhau: plan, retrieve, tool use, tổng hợp
 - ✓ Có thể chia việc độc lập: 2 worker làm song song vẫn hợp lý
 - ✓ Cần debug rõ ai làm sai: route sai, retrieve sai, hay synthesis sai
 - ✓ Cần mở rộng dần: thêm 1 worker mới mà không viết lại cả prompt gốc
 - ✓ Context window một agent không đủ: tài liệu lớn, nhiều tool output
- Đừng dùng multi-agent chỉ vì thấy "ngầu". Nếu 1 workflow đơn giản đã đủ, giữ đơn giản sẽ rẻ và ổn định hơn

02

Mental Model: Tư Duy Hệ Thống Trước Khi Thiết Kế

Trước khi chọn pattern hay tool, cần hình thành tư duy đúng về cách chia trách nhiệm trong một hệ thống phức tạp

Từ "Agent Thông Minh" Sang "Hệ Thống Rõ Ràng"

Tư duy cũ

- Làm thế nào để agent thông minh hơn?
- Prompt nào khiến agent làm được nhiều hơn?
- Thêm nhiều tool cho một agent để đủ sức

Tư duy hệ thống

- Task này gồm bao nhiêu loại trách nhiệm khác nhau?
- Ai cần biết gì, khi nào?
- Lỗi cần được khoanh vùng ở đâu?
- Điểm nào cần human oversight?

Hệ thống clear về vai trò, dễ test từng phần, và dễ cải thiện dần.

Ba Câu Hỏi Thiết Kế Trước Khi Viết Code

Câu hỏi 1

Chia trách nhiệm ở đâu?

Task nào cần reasoning? Task nào cần data fetching? Task nào chỉ cần format?

Câu hỏi 2

Thông tin đi theo con đường nào?

Agent nào cần biết gì? Ai cần đầu ra của ai trước tiên?

Câu hỏi 3

Lỗi xảy ra ở đâu là ít tổn hại nhất?

Thiết kế để lỗi tại worker không làm hỏng toàn bộ hệ thống.

Thiết kế tốt giúp lỗi có địa chỉ rõ ràng thay vì lan ra cả hệ thống.

03

Multi-Agent Patterns

Có nhiều cách chia hệ thống thành nhiều agent; điều quan trọng là chọn pattern giúp giải quyết đúng loại phức tạp, không tạo thêm phức tạp giả

4 Pattern Phổ Biến

Supervisor-Worker

1 supervisor điều phối nhiều worker chuyên biệt.
Mạnh ở: routing rõ, dễ kiểm soát, dễ trace

Pipeline

Agent A xong rồi mới chuyển output cho B.
Mạnh ở: flow ổn định, tuyến tính, dễ test

Debate

Nhiều agent cùng giải một bài toán rồi vote hoặc synthesize.
Mạnh ở: phản biện và giảm blind spot

Hierarchical

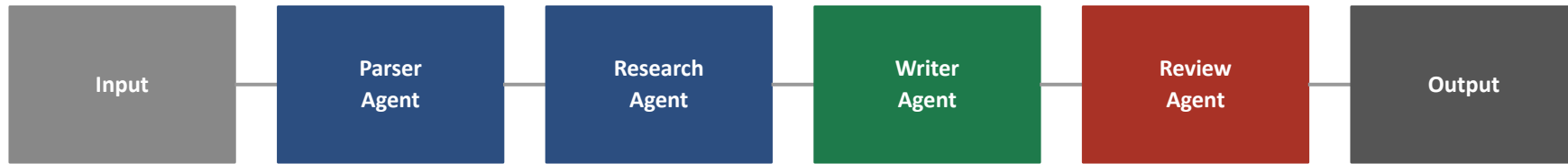
Supervisor lồng supervisor cho nhiều tầng hệ thống.
Mạnh ở: mở rộng tốt ở enterprise scale

Chọn Pattern Theo Loại Bài Toán

Pattern	Dùng khi nào	Điểm mạnh	Cảnh báo
Supervisor-worker	Task cần route tới đúng vai trò	dễ kiểm soát, dễ trace	supervisor có thể thành bottleneck
Pipeline	Các bước gần như cố định	dễ hiểu, dễ test theo bước	kém linh hoạt khi flow đổi động
Debate	Cần nhiều góc nhìn cho cùng một bài toán	giảm blind spot, tăng phản biện	tốn cost và khó tổng hợp
Hierarchical	Nhiều nhóm task và nhiều tầng quản trị	mở rộng tốt ở quy mô lớn	thiết kế và debugging phức tạp

Đừng để 4 pattern ngang nhau trong đầu người học

Minh Họa: Pipeline Pattern Trông Như Thế Nào?



Phù hợp nhất khi

- flow gần như cố định
- mỗi bước cần output của bước trước
- dễ test từng agent riêng biệt

Hạn chế

- latency cộng dồn ở mỗi bước
- khó xử lý khi flow cần rẽ nhánh
- retry một bước ảnh hưởng toàn chuỗi

Minh Họa: Debate Pattern



Khi bài toán có nhiều góc nhìn hợp lệ, khi rủi ro sai cao, hoặc khi cần kiểm tra chéo trước khi ra quyết định quan trọng.

Vì Sao Day 09 Chọn Supervisor-Worker?

Lý do sự phạm

- học viên dễ nhìn ra vai trò
- dễ nối với use case thật
- dễ giải thích logic route
- dễ nâng cấp từ artifact Day 08

Lý do triển khai

- bắt đầu từ 2–3 worker là đủ
- dễ cắm thêm MCP tool worker
- trace và testing rõ hơn
- supervisor thường chỉ cần một LLM call nhỏ

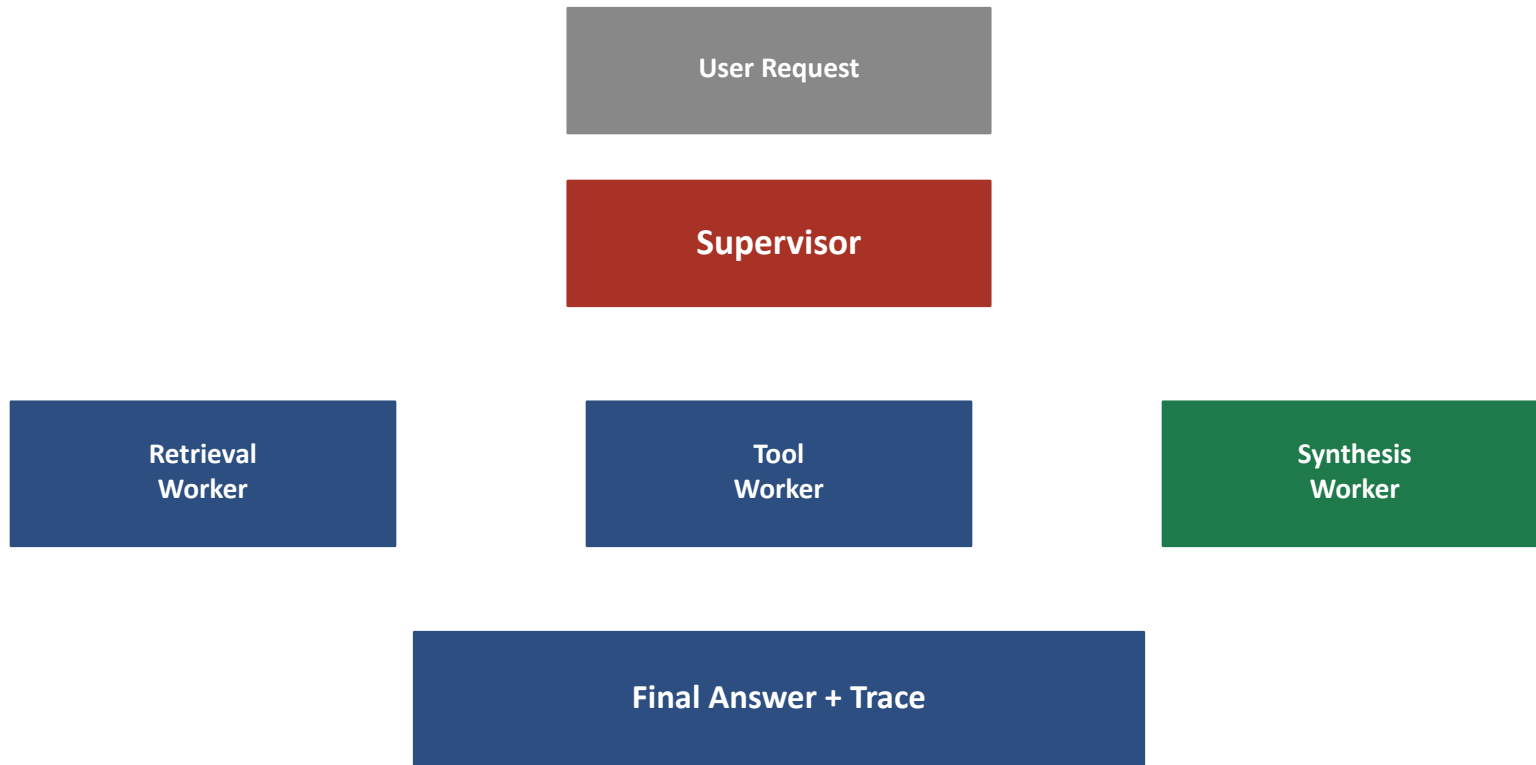
Lưu ý: Nếu Day 09 ôm nhiều pattern ngang nhau, người học sẽ nhớ tên pattern nhưng không biết ngày mai nên build theo pattern nào.

04

Supervisor-Worker: Deep Dive

Thay vì ép một agent làm hết, ta cho một supervisor phân việc và nhiều worker làm phần việc hẹp, dễ kiểm soát hơn

Supervisor-Worker Architecture



Supervisor Làm Gì, Worker Làm Gì?

Supervisor

- phân tích yêu cầu ban đầu
- quyết định worker nào nên tham gia
- theo dõi trạng thái và retry nếu cần
- tổng hợp đầu ra cuối cùng
- biết khi nào cần human review

Worker

- xử lý một năng lực hẹp
- nhận input rõ ràng, trả output rõ ràng
- càng stateless càng dễ test
- thất bại cục bộ không làm hỏng cả kiến trúc
- có thể được thay thế mà không ảnh hưởng supervisor

Supervisor giữ decision flow; worker giữ domain skill. Đừng để một worker vừa làm việc hẹp vừa bí mật điều phối cả hệ thống.

Thiết Kế Worker Tốt Có 3 Đặc Điểm

Tập trung một vai

Một worker nên có một năng lực chính: retrieve, gọi tool, tóm tắt, kiểm tra policy...

Stateless nếu có thể

Nếu có thể, worker chỉ cần input hiện tại thay vì ôm cả lịch sử hệ thống.

Testable độc lập

Có input / output rõ để test độc lập trước khi cắm vào supervisor.

Lưu ý: Worker mơ hồ thường làm debugging cực khó: không biết lỗi do route sai, prompt sai, hay contract đầu vào chưa đủ rõ.

Anti-Pattern: Những Lỗi Thiết Kế Hay Gặp

God Supervisor

Supervisor làm quá nhiều: plan, retrieve, synthesize, monitor. Nó trở thành single-agent được đổi tên.

Implicit State

State bị truyền qua biến toàn cục hoặc side effect. Không ai biết hệ đang ở bước nào.

Chatty Workers

Workers liên tục gọi ngược lại supervisor để hỏi thêm thông tin. Message overhead tăng nhanh.

No Fallback

Worker gặp lỗi và không trả về gì. Supervisor chờ mãi không thấy đầu ra để tổng hợp.

State Schema Tối Thiểu Cho Day 09

Những trường cần có trong shared state:

- task: task gốc từ user
- plan: danh sách worker cần gọi
- worker_results: dict chứa output từng worker
- status: pending | running | done | error
- final_answer: tổng hợp cuối
- trace: log dạng list có timestamp

Tại sao trace là trường bắt buộc?

Không có trace trong state, sau khi hệ chạy xong không ai biết agent đã đi theo con đường nào để ra kết quả đó.

05

MCP — Model Context Protocol

Nếu supervisor-worker là cách chia người làm việc, thì MCP là cách agent cắm được vào năng lực bên ngoài mà không phải custom từng tích hợp từ đầu

MCP Xuất Hiện Để Giải Quyết Vấn Đề Gì?

Trước MCP — vấn đề thực tế

- mỗi tool cần một adapter riêng
- thay đổi API của tool = viết lại code tích hợp
- mỗi framework gọi tool theo cách khác nhau
- không có chuẩn chung để agent biết tool làm gì

MCP giải quyết

- Một chuẩn giao tiếp duy nhất giữa agent và tool
- Agent biết cách "khám phá" các capability mà không cần hard-code từng tích hợp

Điều quan trọng với người học là hiểu vì sao MCP giúp mở rộng hệ thống, không phải học thuộc protocol spec trong buổi đầu tiên.

MCP Là Gì Theo Cách Hiểu Thực Dụng?

- MCP là một chuẩn để agent kết nối với external capabilities.
- Thay vì mỗi tool có một kiểu tích hợp riêng, agent có thể nói chuyện với một MCP server.
- MCP server công bố các thứ như tools, resources, và prompts.
- Agent có thể list, describe, và invoke các capability đó theo chuẩn chung.

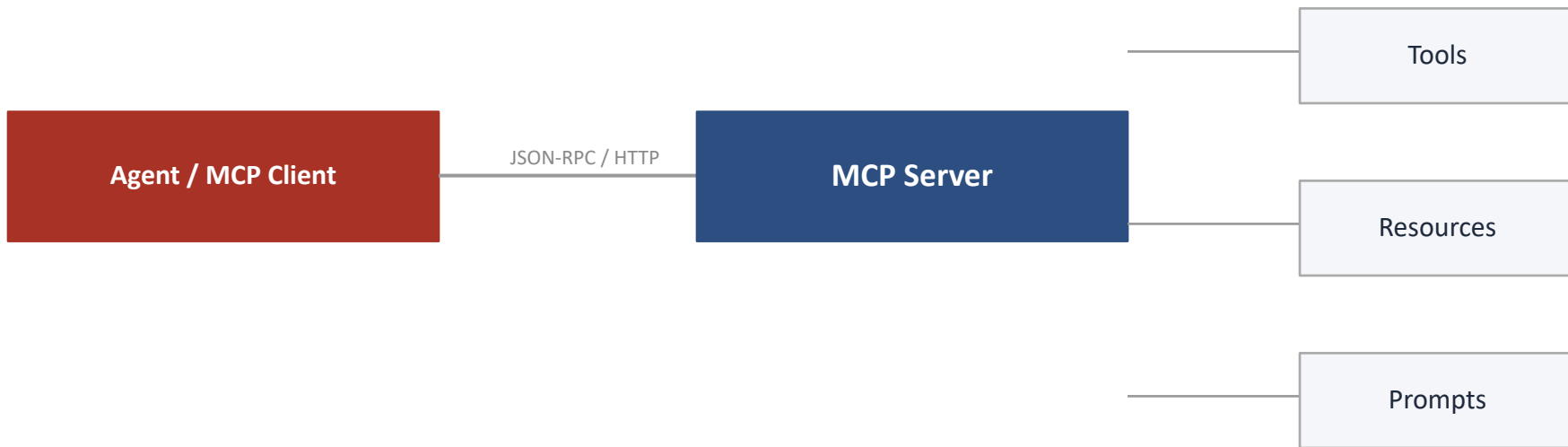
Analogy dễ nhớ

Supervisor-worker nói về vai trò.

MCP nói về ổ cắm chuẩn để agent dùng tài nguyên bên ngoài.

Giống USB: mọi thiết bị cùng dùng một chuẩn kết nối.

MCP Architecture



Agent không cần biết chi tiết từng hệ thống phía sau. Nó chỉ cần hiểu MCP surface mà server công bố.

MCP Tool Discovery: Agent Tìm Hiểu Tool Như Thế Nào?

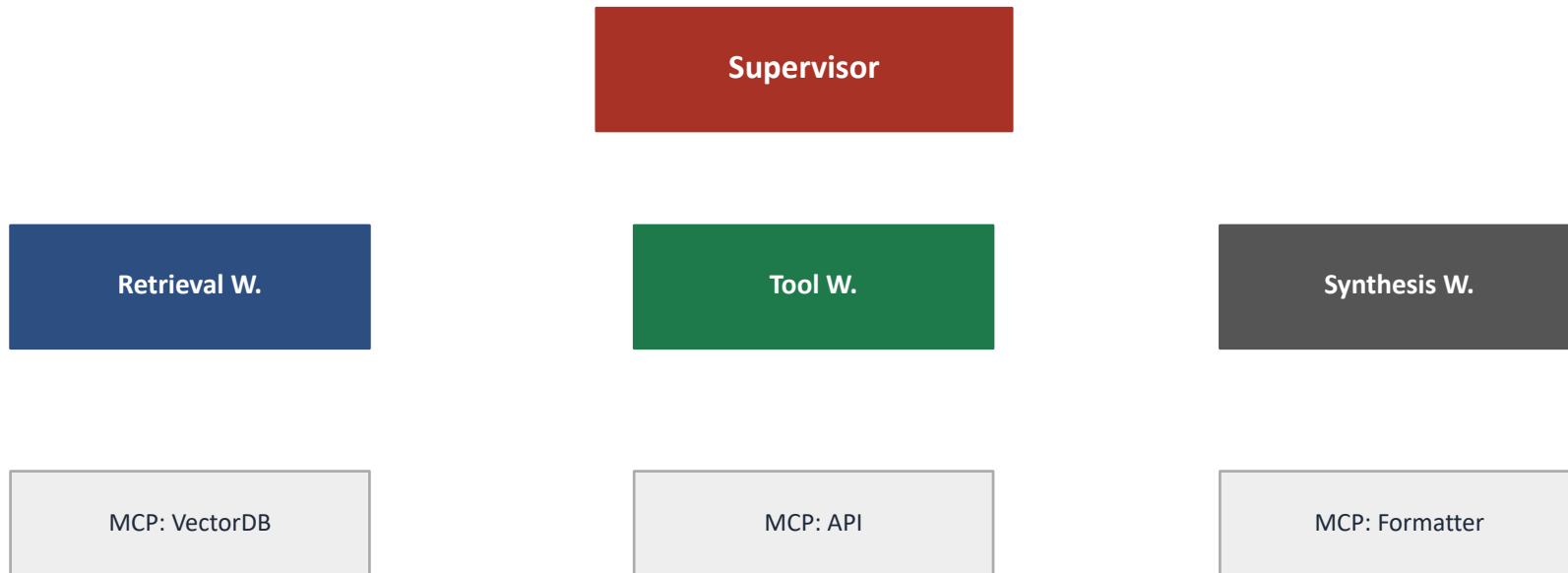
Luồng discovery

1. Agent kết nối tới MCP server
2. Gọi tools/list để lấy danh sách tool
3. Mỗi tool trả về: name, description, inputSchema
4. Agent đọc schema, quyết định tool nào phù hợp
5. Gọi tools/call với đúng parameters
6. MCP server thực thi và trả kết quả

Tại sao quan trọng?

Agent không cần được lập trình sẵn biết tool "X" tồn tại. Nó có thể khám phá khi chạy và tự điều chỉnh theo tool nào có sẵn.

MCP Trong Bức Tranh Toàn Hệ Thống



MCP là lớp giữa worker và capability thực. Worker chỉ cần biết MCP surface, không cần biết hệ thống phía sau là gì.

06

A2A — Agent to Agent Communication

MCP giúp agent nói chuyện với tool; A2A giúp agent nói chuyện với agent khác theo cách rõ nhiệm vụ, rõ bối cảnh, và rõ đầu ra mong đợi

Đừng Nhầm MCP Với A2A

MCP

- agent nói chuyện với tool / capability
- mục tiêu là kết nối năng lực bên ngoài
- trọng tâm là surface chuẩn
- tool không có agency — chỉ thực thi

A2A

- agent nói chuyện với agent khác
- mục tiêu là chia việc và đồng bộ
- trọng tâm là message contract rõ ràng
- agent phía kia có thể ra quyết định

Cùng là "gọi ra ngoài", nhưng MCP trả lời câu hỏi agent lấy năng lực ở đâu, còn A2A trả lời câu hỏi agent giao việc cho ai.

Một Message Contract Tối Thiểu Cho A2A

Task

Agent kia cần làm gì?
Ví dụ: tìm 3 chunk policy phù hợp nhất

Context

Những gì worker cần biết để làm đúng?
query, constraints, user role, state

Expected Output

Trả về theo format nào?
list chunks, score, rationale, error

Ví dụ:

```
task = "retrieve evidence"
```

```
context = "user hỏi về refund policy, ưu tiên tài liệu sau 2025"
```

```
expected_output = "top 3 chunks + source + confidence"
```

07

Orchestration Với LangGraph

Sau khi hiểu vai trò và giao tiếp, ta cần một cách biểu diễn luồng chạy rõ ràng; LangGraph là cách rất trực quan để làm điều đó

Tại Sao Cần Orchestration Framework?

Không có framework

- routing logic nằm trong prompt điều phối dài
- khó biết hệ đang ở bước nào
- thêm một nhánh mới = sửa toàn bộ prompt
- debug bằng print() và hy vọng

Với LangGraph

- Routing trở thành code tường minh
- Graph có thể visualize
- State có schema
- Human-in-the-loop có điểm rõ ràng

Nếu không có orchestration rõ ràng, routing logic thường bị chôn trong prompt và trở nên rất khó debug.

LangGraph Đóng Vai Trò Gì?

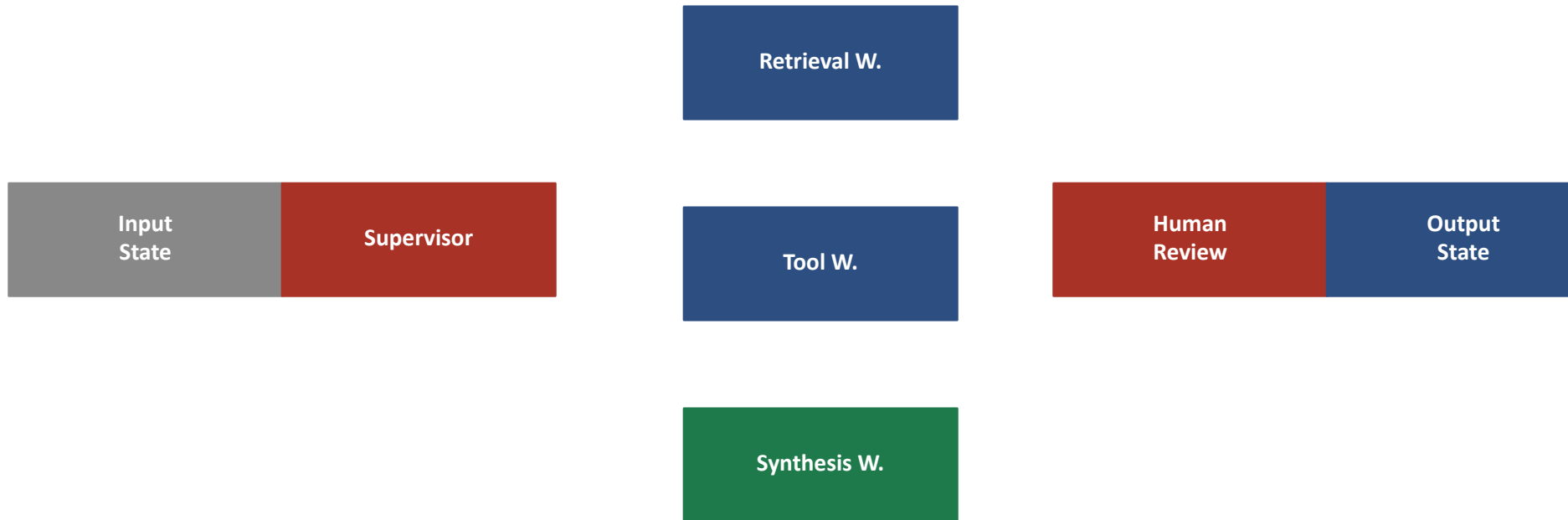
- Biến hệ multi-agent thành graph gồm nodes, edges, và state.
- Tách rõ node nào làm việc gì và khi nào route sang node nào.
- Giúp hệ thống bớt phụ thuộc vào một prompt điều phối khổng lồ.
- Hỗ trợ persistence: state có thể được lưu và tiếp tục.
- Hỗ trợ human-in-the-loop tại bất kỳ điểm nào.

Khung nhớ nhanh

node = ai làm
edge = đi đâu tiếp
state = hệ đang biết gì

Ba khái niệm này là toàn bộ LangGraph bạn cần cho Day 09.

LangGraph Routing Diagram



LangGraph làm lộ rõ route quyết định ở đâu, state đi qua đâu, và human can thiệp ở điểm nào.

Ví Dụ Routing Logic

```
class AgentState(TypedDict):
    task: str
    need_retrieval: bool
    need_tool: bool
    worker_results: dict
    final_answer: str

def route_to_worker(state: AgentState) -> str:
    if state["need_tool"]:
        return "tool_worker"
    if state["need_retrieval"]:
        return "retrieval_worker"
    return "synthesis_worker"

graph.add_conditional_edges(
    "supervisor",
    route_to_worker,
    {
        "tool_worker": "tool_worker",
        "retrieval_worker": "retrieval_worker",
        "synthesis_worker": "synthesis_worker",
    }
)
```

Human-in-the-Loop Đặt Ở Đâu?

Nên chèn khi

- task có rủi ro cao (tài chính, y tế, pháp lý)
- confidence score dưới ngưỡng
- tool action có side effect không đảo ngược
- output sẽ đi ra user hoặc stakeholder

Cách implement trong LangGraph

- thêm node "human_review" vào graph
- node này interrupt graph và chờ input
- sau khi human approve → tiếp tục
- state được giữ nguyên qua interrupt

Multi-agent không đồng nghĩa với full autonomy. Nhiều hệ tốt nhất là hệ biết khi nào nên dừng để con người quyết định.

08

Observability & Debugging Hệ Multi-Agent

Hệ thống nhiều agent khó debug hơn nhiều so với single agent. Observability tốt là điều kiện tiên quyết để cải thiện được hệ thống

Vì Sao Multi-Agent Khó Debug Hơn?

Nguồn gốc lỗi khó xác định

- Lỗi có thể xuất phát từ: routing sai, context sai, tool fail, synthesis sai
- Lỗi ở bước A có thể chỉ lộ ra ở bước C
- Nhiều agent = nhiều LLM call = nhiều điểm fail tiềm năng

Lưu ý: Không có trace tốt, debugging multi-agent gần như là mò mẫm.

3 câu hỏi observability

1. Agent nào đã chạy, theo thứ tự nào?
2. Input/output tại mỗi bước là gì?
3. Lỗi hay warning nào đã xảy ra?

Thiết Kế Trace Log Tốt

Mỗi entry trong trace nên có

- timestamp: khi nào
- agent_id: ai làm
- action: làm gì (route / call / synthesize / error)
- input_summary: nhận gì (tóm tắt)
- output_summary: trả về gì
- status: ok | warn | error
- latency_ms: mất bao lâu

Ví dụ trace entry

```
{
  "t": "14:03:21",
  "agent": "supervisor",
  "action": "route",
  "decision": "retrieval_worker",
  "reason": "need_retrieval=true",
  "status": "ok",
  "latency_ms": 312
}
```

Từ Trace Đến Cải Thiện Hệ Thống



Trace không chỉ để debug lỗi hôm nay. Nó là dữ liệu để cải thiện routing, worker quality, và message contract theo thời gian.

09

Cost, Latency & Reliability

Multi-agent không miễn phí. Nhiều agent = nhiều LLM call = nhiều tiền và nhiều điểm fail. Cần tư duy trade-off từ sớm

So Sánh: Single vs Multi-Agent

Tiêu chí	Single Agent	Multi-Agent
Chi phí LLM	Thấp hơn (1 call)	Cao hơn (nhiều call)
Latency	Thấp nếu prompt ngắn	Có thể song song; tăng nếu serial
Debuggability	Khó hơn (logic ẩn)	Tốt hơn (có trace rõ)
Specialization	Hạn chế	Tốt hơn (mỗi worker chuyên biệt)
Scalability	Khó scale vai trò	Dễ thêm worker mới
Complexity	Đơn giản hơn	Phức tạp hơn khi setup

Trade-off quan trọng cần hiểu khi thiết kế

Reliability: Khi Worker Thất Bại

Cần thiết kế trước

- Worker có timeout rõ ràng
- Supervisor có retry logic: thử lại bao nhiêu lần?
- Nếu retry cũng fail: fallback là gì?
- Thất bại cục bộ không nên crash toàn bộ hệ thống
- Partial failure: tổng hợp kết quả với những worker đã thành công

Graceful degradation

Hệ thống không cần làm tốt mọi trường hợp. Nó cần thất bại theo cách kiểm soát được và báo cáo rõ ràng khi không đủ tự tin.

01

Lab 9: Multi-Agent System + MCP

Biến một agent RAG đơn thành một hệ multi-agent nhỏ có route rõ, capability rõ, và trace rõ để dễ giải thích và debug hơn

Lab 9: Các Bước Thực Hiện

Biến một agent RAG đơn thành một hệ multi-agent nhỏ có route rõ, capability rõ, và trace rõ để dễ giải thích và debug hơn.

1. Tách artifact Day 08 thành supervisor + 2–3 workers
2. Thiết kế shared state schema với trường trace
3. Chọn 1 worker dùng external capability qua MCP
4. Viết message contract tối thiểu giữa supervisor và workers
5. Trace lại toàn bộ luồng để biết agent nào đã làm gì
6. Demo kết quả cuối cùng kèm reasoning flow ở mức quan sát được

Rubric Đánh Giá Lab 9

Tiêu chí	Đạt (70%)	Tốt (85%)	Xuất sắc (100%)
Phân vai trò	Có supervisor + 2 workers	Vai trò rõ, không overlap	Anti-patterns được tránh có ý thức
MCP kết nối	Có 1 MCP call hoạt động	Schema rõ, trace ghi được	Discovery đúng, error handling có
Shared state	State hoạt động	Schema đầy đủ, trace field	Ownership rõ từng field
Trace quality	Log cơ bản	Đủ 5 trường cần thiết	Actionable, dẫn đến insight
Routing logic	Routing hoạt động	Conditional edge rõ	Giải thích được quyết định route

Tổng Kết — Key Takeaways

1

Multi-agent là cách chia vai trò để hệ thống đỡ quá tải và dễ kiểm soát hơn, không phải chỉ là tăng số lượng agent. Chỉ dùng khi bài toán thực sự cần.

2

Supervisor-worker là pattern practical nhất để bắt đầu. Supervisor route và tổng hợp; worker chuyên một năng lực hẹp, stateless, testable.

3

MCP cho agent cắm vào tool qua chuẩn chung. A2A cho agents giao việc cho nhau với message contract rõ. Hai thứ này giải quyết hai vấn đề khác nhau.

4

LangGraph biến orchestration thành graph có state và conditional routing — dễ debug, dễ thêm human-in-the-loop, và dễ visualize luồng quyết định.

5

Observability là điều kiện tiên quyết. Trace tốt không chỉ để debug hôm nay — nó là dữ liệu để cải thiện hệ thống theo thời gian.

Tài Liệu Tham Khảo

1. Model Context Protocol, Official Documentation — modelcontextprotocol.io — client / server model, tools, resources, prompts.
2. LangGraph Docs, Multi-Agent Tutorials — supervisor-worker orchestration, graph routing, state handling, human-in-the-loop.
3. Sumers et al. (2023), Cognitive Architectures for Language Agents (CoALA) — phân loại memory, action, và decision trong language agents.
4. Anthropic (2024), Building Effective Agents — blog post về practical patterns cho agentic systems.
5. LangSmith Documentation, Tracing & Observability — công cụ trace cho LangChain/LangGraph pipelines.

Hỏi & Đáp

"Một agent rất giỏi có thể làm nhiều việc. Nhưng hệ thống tốt hơn thường bắt đầu từ câu hỏi: nên chia vai trò ở đâu để dễ kiểm soát và dễ cải thiện nhất?"